

LAPORAN PRAKTIKUM 9
SISTEM BASIS DATA



Disusun Oleh :

Muhammad Harits Arrozzaq (2400016087)

Kelas Praktikum/Semester :

B/III

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS SAINS & TEKNOLOGI TERAPAN
UNIVERSITAS AHMAD DAHLAN
YOGYAKARTA
2025

A. Dasar Teori

Transaksi dalam RDBMS

Transaksi adalah satu unit kerja logis yang berisi satu atau lebih perintah SQL dan harus dieksekusi secara utuh agar data tetap benar.

Prinsip ACID

1. Atomicity (Keutuhan)

- All or Nothing
- Jika satu perintah gagal, seluruh transaksi dibatalkan (ROLLBACK)
- Contoh: INSERT pesanan berhasil tapi UPDATE stok gagal → INSERT ikut dibatalkan

2. Consistency (Konsistensi)

- Data harus selalu memenuhi aturan/constraint
- Contoh: stok tidak boleh negatif
- Jika transaksi melanggar aturan → transaksi gagal

3. Isolation (Isolasi)

- Transaksi yang berjalan bersamaan tidak saling mengganggu
- Masalah umum:
 - a) Dirty Read: baca data belum commit
 - b) Non-Repeatable Read: data berubah saat dibaca ulang
 - c) Phantom Read: jumlah baris berubah karena insert baru

4. Durability (Daya Tahan)

- Setelah COMMIT, data tersimpan permanen
- Tetap aman walau sistem mati mendadak
- MySQL menggunakan Write-Ahead Logging (WAL)

Syntax Referensi

Perintah	Fungsi
START TRANSACTION	Memulai transaksi
COMMIT	Menyimpan perubahan
ROLLBACK	Membatalkan perubahan
SAVEPOINT nama_point	Titik rollback sebagian
ROLLBACK TO nama_point	Kembali ke savepoint
SELECT ... FOR UPDATE	Mengunci data saat dibaca

B. Langkah – Langkah Percobaan

Persiapan Studi Kasus (E-Commerce)

a. Membuat Database

```
mysql> CREATE DATABASE ecommerce_db;  
Query OK, 1 row affected (0.02 sec)  
  
[mysql> USE ecommerce_db;  
Database changed
```

b. Buat Tabel produk

```
[mysql> CREATE TABLE produk (  
[   -> id INT AUTO_INCREMENT PRIMARY KEY,  
[   -> nama_barang VARCHAR(100),  
[   -> stok INT CHECK (stok >= 0)  
[   -> ) ENGINE=InnoDB;  
Query OK, 0 rows affected (0.02 sec)
```

c. Buat Tabel pesanan

```
[mysql> CREATE TABLE pesanan (  
[   -> id INT AUTO_INCREMENT PRIMARY KEY,  
[   -> nama_pembeli VARCHAR(100),  
[   -> id_produk INT,  
[   -> jumlah INT,  
[   -> status VARCHAR(20) DEFAULT 'Pending',  
[   -> waktu_pesan TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
[   -> FOREIGN KEY (id_produk) REFERENCES produk(id)  
[   -> ) ENGINE=InnoDB;  
Query OK, 0 rows affected (0.01 sec)
```

d. Seeding Data

```
[mysql> INSERT INTO produk (nama_barang, stok) VALUES ('iPhone 15 Pro', 5);  
Query OK, 1 row affected (0.02 sec)  
  
mysql> INSERT INTO produk (nama_barang, stok) VALUES ('Samsung S24', 10);  
Query OK, 1 row affected (0.00 sec)
```

Langkah Praktikum

Skenario 1: Atomicity (Pembelian Normal vs Error)

Tujuan: Memastikan stok berkurang HANYA JIKA pesanan berhasil dicatat.

Percobaan A: Transaksi Sukses

a. START TRANSACTION;

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

b. Catat Pesanan

```
[mysql> INSERT INTO pesanan (nama_pembeli, id_produk, jumlah, status) VALUES  
[    -> ('User_A', 1, 1, 'Success');  
Query OK, 1 row affected (0.00 sec)
```

c. Kurangi Stok

```
[mysql> UPDATE produk SET stok = stok - 1 WHERE id = 1;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```

d. Cek hasil sementara

```
[mysql> SELECT * FROM produk;  
+-----+-----+-----+  
| id | nama_barang | stok |  
+-----+-----+-----+  
| 1 | iPhone 15 Pro | 4 |  
| 2 | Samsung S24 | 10 |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

```
[mysql> SELECT * FROM pesanan;  
+-----+-----+-----+-----+-----+-----+  
| id | nama_pembeli | id_produk | jumlah | status | waktu_pesan |  
+-----+-----+-----+-----+-----+-----+  
| 2 | User_A | 1 | 1 | Success | 2025-12-17 09:45:26 |  
+-----+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

e. Simpan permanen

```
[mysql> COMMIT;  
Query OK, 0 rows affected (0.00 sec)
```

Percobaan B: Simulasi Crash/Error (Rollback)

User membeli 2 iPhone, tapi sistem error setelah insert pesanan.

a. **START TRANSACTION;**

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

b. **Catat pesanan**

```
[mysql> INSERT INTO pesanan (nama_pembeli, id_produk, jumlah, status) VALUES  
[    -> ('User_B', 1, 2, 'Pending');  
Query OK, 1 row affected (0.00 sec)
```

c. **Ceritanya di sini koneksi putus atau program crash sebelum update stok**

```
[mysql> ROLLBACK;  
Query OK, 0 rows affected (0.01 sec)
```

d. **Verifikasi**

```
[mysql> SELECT * FROM pesanan WHERE nama_pembeli = 'User_B';  
Empty set (0.00 sec)
```

e. **Stok harus Kembali ke 4**

```
[mysql> SELECT * FROM produk WHERE id = 1;  
+----+-----+-----+  
| id | nama_barang | stok |  
+----+-----+-----+  
| 1  | iPhone 15 Pro | 4    |  
+----+-----+-----+  
1 row in set (0.00 sec)
```

Skenario 2: Consistency (Menjaga Aturan Bisnis)

Tujuan: Mencegah stok minus saat pembelian melebihi persediaan.

a. START TRANSACTION;

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

b. Coba update stok (4 - 10 = -6)

```
[mysql> UPDATE produk SET stok = stok - 10 WHERE id = 1;  
ERROR 3819 (HY000): Check constraint 'produk_chk_1' is violated.
```

c. Karena error, kita tidak boleh lanjut insert pesanan.

```
[mysql> ROLLBACK;  
Query OK, 0 rows affected (0.00 sec)
```

Analisis: Database menjaga **Konsistensi** data dengan menolak input yang melanggar aturan logika (stok minus).

Skenario 3: Isolation & Locking (Simulasi "War Tiket")

Tujuan: Mencegah Race Condition di mana 2 orang membeli barang terakhir secara bersamaan.

Langkah 1: Sesi 1 melihat stok dan bersiap beli (tapi lelet)

a. START TRANSACTION;

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

b. Sesi 1 mengecek stok (belum dikunci)

```
[mysql> SELECT * FROM produk WHERE id = 1;  
+----+-----+-----+  
| id | nama_barang | stok |  
+----+-----+-----+  
| 1  | iPhone 15 Pro | 4    |  
+----+-----+-----+  
1 row in set (0.00 sec)
```

Langkah 2: Sesi 2 menyerobot beli dan commit duluan

- a. **START TRANSACTION;**

```
[mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

- b. **UPDATE produk SET stok = stok - 2 WHERE id = 1;**

```
[mysql> UPDATE produk SET stok = stok - 2 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- c. **COMMIT;**

```
[mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

- d. Sesi 2 sukses beli 2 unit. Stok asli di DB sekarang tinggal 2.

Langkah 3: Sesi 1 melanjutkan proses (Masalah Terjadi)

- a. **Update mengurangi 2 unit juga.**

```
[mysql> UPDATE produk SET stok = stok - 2 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- b. **COMMIT;**

```
[mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

- c. Stok menjadi 0. Secara teknis benar, tapi dalam aplikasi *real-time*, Sesi 1 bekerja berdasarkan data kadaluarsa.

```
[mysql> SELECT * FROM produk WHERE id = 1;
+----+-----+-----+
| id | nama_barang | stok |
+----+-----+-----+
|  1 | iPhone 15 Pro |    0 |
+----+-----+-----+
1 row in set (0.00 sec)
```


Solusi: Pessimistic Locking (FOR UPDATE)

1. Sesi 1:

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

2. Sesi 1:

```
[mysql> SELECT * FROM produk WHERE id = 1 FOR UPDATE;  
+-----+-----+-----+  
| id | nama_barang | stok |  
+-----+-----+-----+  
| 1 | iPhone 15 Pro | 4 |  
+-----+-----+-----+  
1 row in set (0.00 sec)
```

3. Sesi 2:

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

4. Sesi 2:

```
[mysql> SELECT * FROM produk WHERE id = 1 FOR UPDATE;  
█
```

5. Sesi 1:

```
[mysql> UPDATE produk SET stok = stok - 1 WHERE id = 1;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1 Changed: 1 Warnings: 0  
  
[mysql> COMMIT;  
Query OK, 0 rows affected (0.00 sec)
```


Skenario 4: Savepoint (Transaksi Bertingkat)

Tujuan: Membatalkan sebagian langkah tanpa membatalkan seluruh transaksi.

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

1. Beli iPhone (Sukses)

```
mysql> INSERT INTO pesanan (nama_pembeli, id_produk, jumlah) VALUES ('User_C', 1, 1);  
Query OK, 1 row affected (0.00 sec)  
  
mysql> UPDATE produk SET stok = stok - 1 WHERE id = 1;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1 Changed: 1 Warnings: 0
```

2. Pasang Titik Aman

```
[mysql> SAVEPOINT beli_iphone_sukses;  
Query OK, 0 rows affected (0.00 sec)
```

3. Coba beli Samsung (Tapi User berubah pikiran/Error)

```
mysql> INSERT INTO pesanan (nama_pembeli, id_produk, jumlah) VALUES ('User_C', 2, 11);  
Query OK, 1 row affected (0.00 sec)  
  
mysql> ROLLBACK TO SAVEPOINT beli_iphone_sukses;  
Query OK, 0 rows affected (0.00 sec)
```

4. Finalisasi

```
[mysql> COMMIT;  
Query OK, 0 rows affected (0.00 sec)
```

5. Cek Data

```
mysql> SELECT * FROM pesanan WHERE nama_pembeli = 'User_C';  
+----+-----+-----+-----+-----+-----+  
| id | nama_pembeli | id_produk | jumlah | status | waktu_pesan |  
+----+-----+-----+-----+-----+-----+  
| 4 | User_C | 1 | 1 | Pending | 2025-12-17 10:33:03 |  
+----+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

C. Tugas Praktikum

Tugas 1: The Deadlock Challenge

1. **Terminal A:** Mulai transaksi, update stok iPhone. (Jangan Commit).

```
[mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

```
[mysql> UPDATE produk SET stok = stok - 1 WHERE id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

2. **Terminal B:** Mulai transaksi, update stok Samsung. (Jangan Commit).

```
[mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

[mysql> UPDATE produk SET stok = stok - 1 WHERE id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

3. **Terminal A:** Update stok Samsung. (Loading menunggu Terminal B).

```
[mysql> UPDATE produk SET stok = stok - 1 WHERE id = 2;
```

4. **Terminal B:** Update stok iPhone.

```
[mysql> UPDATE produk SET stok = stok - 1 WHERE id = 1;
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

Pertanyaan: Apa pesan error yang muncul di MySQL? Jelaskan dengan bahasamu sendiri kenapa ini terjadi!

Jawaban: Error ini terjadi karena dua transaksi saling mengunci data yang dibutuhkan satu sama lain. Terminal A mengunci data iPhone lalu menunggu Samsung yang dikunci Terminal B, sementara Terminal B menunggu iPhone yang dikunci Terminal A. Karena keduanya saling menunggu dan tidak bisa lanjut, MySQL mendeteksi kondisi deadlock dan membatalkan salah satu transaksi agar sistem tidak macet.

Tugas 2: Dirty Read Simulation

1. **Ubah Isolation Level di terminal Anda menjadi READ UNCOMMITTED.**

```
[mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.01 sec)
```

2. **Lakukan simulasi di mana Terminal A melakukan update stok tapi belum commit**

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE produk SET stok = 0 WHERE id = 1;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

```

3. Terminal B (mode READ UNCOMMITTED) melakukan SELECT.

```

mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.01 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT stok FROM produk WHERE id = 1;
+-----+
| stok |
+-----+
|    0 |
+-----+
1 row in set (0.00 sec)

```

Pertanyaan: Apakah Terminal B melihat stok baru atau stok lama? Mengapa kondisi ini berbahaya bagi laporan keuangan perusahaan?

Jawaban: Terminal B melihat stok baru meskipun belum di-commit karena isolation level READ UNCOMMITTED mengizinkan pembacaan data sementara. Kondisi ini berbahaya karena data tersebut bisa dibatalkan (rollback), sehingga laporan keuangan berisiko tidak akurat dan dapat menyesatkan pengambilan keputusan bisnis.

Tugas 3: Implementasi Logic

1. User D ingin memborong semua stok Samsung.

```

mysql> INSERT INTO pesanan (nama_pembeli, id_produk, jumlah)
      -> SELECT 'User_D', id, stok
      -> FROM produk
      -> WHERE id = 2 AND stok > 0;
Query OK, 1 row affected (0.01 sec)
Records: 1  Duplicates: 0  Warnings: 0

```

2. Sistem harus mengecek sisa stok dulu.

```
[mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

[mysql> SELECT stok FROM produk WHERE id = 2 FOR UPDATE;
+-----+
| stok |
+-----+
|    9 |
+-----+
1 row in set (0.00 sec)
```

3. Jika stok > 0, buat pesanan dan nol-kan stok.

```
[mysql> UPDATE produk SET stok = 0 WHERE id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

[mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

4. Gunakan FOR UPDATE untuk memastikan keamanan data saat pengecekan.

```
[mysql> SELECT stok FROM produk WHERE id = 2 FOR UPDATE;
+-----+
| stok |
+-----+
|    0 |
+-----+
1 row in set (0.00 sec)
```

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan transaksi dalam sistem basis data sangat penting untuk menjaga keutuhan dan keandalan data. Prinsip ACID memastikan setiap transaksi berjalan secara utuh (atomic), konsisten terhadap aturan yang berlaku, terisolasi dari transaksi lain yang berjalan bersamaan, serta tersimpan secara permanen setelah dilakukan commit. Melalui simulasi atomicity, consistency, isolation, deadlock, dirty read, dan penggunaan FOR UPDATE, dapat dipahami bahwa pengelolaan transaksi yang tepat mampu mencegah masalah seperti stok tidak sesuai, race condition, dan data laporan yang tidak akurat. Oleh karena itu, penerapan mekanisme transaksi dan locking sangat dibutuhkan dalam sistem e-commerce agar data tetap aman dan konsisten.